needs the web server as a data store and to execute certain kinds of business logic, and this part of the architecture can be done with Django.

In a Django-based SPA, a view doesn't render HTML; instead, it serves data to the SPA. This is typically done using JSON. Plugin libraries like the *Django REST Framework* and *Django Ninja* provide features for building APIs based on views as well as tools for serializing your ORM data objects into a form the JavaScript application can consume.

The world isn't binary. There are degrees between a traditional server-side application and a full single-page application. Using libraries like Vue.js, to trigger API calls, or HTMX, which uses pieces of HTML, you can augment server-side rendering with client-side flexibility. Django fits this naturally, with views rendering full pages, supporting APIs, and HTML snippets, as required.

1.2.2 When and where you should use it

There are lots of choices in the Python world for writing web applications. A popular alternative to Django is *Flask*. Flask is smaller than Django and only directly offers the routing and view portion. Flask tends to require other libraries to build out a full system. It is often paired with SQLAlchemy for an ORM and Jinja2, for templating. By contrast, Django comes with an ORM and templating engine already. You may be able to get going with Flask a little faster than Django, but if your application grows, you're going to need the other pieces as well. With the routing, views, template rendering, and ORM all coming in one package, you don't have to worry about how and whether things will work together.

The flip side of this argument is that Django isn't really meant to be used in pieces. Although the ORM is fantastic, if you're not building a web application and only want the ORM, Django isn't the best fit. Django is intended to build web projects. You can use it as a tool for other things, but you'll find you're fighting it.

SPAs are designed with a backend server that only provides data, while the GUI component gets handled by the browser. The FastAPI library specializes in providing REST APIs and is a popular choice if you are using Python for the backend of an SPA.

If you are writing SPAs, there is a strong argument for using Node.js on the server side. Having JavaScript as both the client and server language has advantages. If you write your server in Python, you're going to have to duplicate some code, usually code for object to data conversion. Personally, my preference for Python far outweighs this extra work.

Django is sometimes viewed as a server-side only technology because that was what it was originally written for. The URL-maps-to-views approach has come full circle. Originally, it was how you built web pages. Then, single-page applications came along, and this approach was considered older tech. The same site structure works well for APIs though. This means Django can be used as a backend for single-page applications, as the server component for mobile apps, and when machine-to-machine communication is important. The availability of libraries like the Django Rest Framework and Django Ninja (heavily inspired by FastAPI) make writing APIs with Django relatively quick and painless. Being able to have both web pages and APIs in the same